

Continuous Integration

Continuous Delivery

CPython

CI/CD на примере CPython

*Как крупнейший open-source проект управляет
качеством кода на протяжении десятилетий*

Что такое CI/CD?

Continuous Integration

Автоматическая сборка и тестирование
каждого изменения кода сразу после
его добавления в репозиторий

Что такое CI/CD?

Continuous Integration

Автоматическая сборка и тестирование каждого изменения кода сразу после его добавления в репозиторий

Continuous Delivery/Deployment

Автоматизированный процесс доставки протестированного кода в production-среду с минимальным ручным вмешательством.

- *Delivery*: развертывание на сервере вручную
- *Deployment*: развертывание на сервере автоматически

Что такое CI/CD?

Continuous Integration

Автоматическая сборка и тестирование каждого изменения кода сразу после его добавления в репозиторий

Continuous Delivery/Deployment

Автоматизированный процесс доставки протестированного кода в production-среду с минимальным ручным вмешательством.

- *Delivery*: развертывание на сервере вручную
- *Deployment*: развертывание на сервере автоматически

Зачем это нужно?

для автоматизации сборки, тестирования и доставки кода, что ускоряет релизы, снижает количество ошибок и обеспечивает высокое качество программного обеспечения

Почему CPython?

Высокие требования

Более 1,5 млн строк кода на C и Python + тысячи тестов. Где цена ошибки очень высока

Почему CPython?

Высокие требования

Более 1,5 млн строк кода на C и Python + тысячи тестов. Где цена ошибки очень высока

Мультиплатформенность

Python работает на Linux, macOS, Windows, iOS, Android, FreeBSD и WASM — каждая платформа должна быть проверена

Почему CPython?

Высокие требования

Более 1,5 млн строк кода на C и Python + тысячи тестов. Где цена ошибки очень высока

Мультиплатформенность

Python работает на Linux, macOS, Windows, iOS, Android, FreeBSD и WASM — каждая платформа должна быть проверена

Открытые PR от сообщества

Тысячи PR от незнакомых контрибьюторов — автоматические проверки защищают от случайных и намеренных поломок

Почему CPython?

Высокие требования

Более 1,5 млн строк кода на C и Python + тысячи тестов. Где цена ошибки очень высока

Мультиплатформенность

Python работает на Linux, macOS, Windows, iOS, Android, FreeBSD и WASM — каждая платформа должна быть проверена

Открытые PR от сообщества

Тысячи PR от незнакомых контрибьюторов — автоматические проверки защищают от случайных и намеренных поломок

Несколько открытых веток

3.12, *3.13* и *main* поддерживаются параллельно — изменение в одной ветке не должно ломать другие

Архитектура

GitHub actions

Основной CI для pull request'ов. Проверяет сборку, запускает юнит-тесты, линтеры и статический анализ.

- Каждый PR + push
- Lint
- Build + тесты на 3 ОС
- Проверка документации

Архитектура

GitHub actions

Основной CI для pull request'ов. Проверяет сборку, запускает юнит-тесты, линтеры и статический анализ.

- Каждый PR + push
- Lint
- Build + тесты на 3 ОС
- Проверка документации

BuildBot

Распределённая сеть машин, поддерживаемых сообществом. Запускают тяжелые и долгие тесты на огромном количестве платформ и архитектур

- После merge в main()
- Долгие stress-тесты
- ARM, RISC-V, PowerPC
- Debug + Release билды

Архитектура

GitHub actions

Основной CI для pull request'ов. Проверяет сборку, запускает юнит-тесты, линтеры и статический анализ.

- Каждый PR + push
- Lint
- Build + тесты на 3 ОС
- Проверка документации

BuildBot

Распределённая сеть машин, поддерживаемых сообществом. Запускают тяжелые и долгие тесты на огромном количестве платформ и архитектур

- После merge в main()
- Долгие stress-тесты
- ARM, RISC-V, PowerPC
- Debug + Release билды

CPython сочетает скорость современных GitHub Actions и надежность традиционных Buildbots.

Workflows

это пайплайн, который описывает, что, когда и как должен делать GitHub Actions

Это обычный YAML-файл, который лежит в папке `.github/workflows/` и автоматически запускается по определённым событиям (push, pull_request)

Workflows

это пайплайн, который описывает, что, когда и как должен делать GitHub Actions

Это обычный YAML-файл, который лежит в папке `.github/workflows/` и автоматически запускается по определённым событиям (push, pull_request)

```
name: Название workflow

# Когда запускать
on:
  pull_request:
  push:
  workflow_dispatch:

# Права доступа
permissions:
  contents: read

# Управление одновременными запусками
concurrency:
  group: ...
  cancel-in-progress: true

# Переменные окружения
env:
  PYTHON_VERSION: '3.13'
```

Workflows

это пайплайн, который описывает, что, когда и как должен делать GitHub Actions

Это обычный YAML-файл, который лежит в папке `.github/workflows/` и автоматически запускается по определённым событиям (push, pull_request)

```
name: Название workflow

# Когда запускать
on:
  pull_request:
  push:
  workflow_dispatch:

# Права доступа
permissions:
  contents: read

# Управление одновременными запусками
concurrency:
  group: ...
  cancel-in-progress: true

# Переменные окружения
env:
  PYTHON_VERSION: '3.13'
```

```
# Основная часть
jobs:
  job_name:
    name: Описание джоба
    runs-on: ubuntu-latest
    timeout-minutes: 30

# Матрица (множество комбинаций)
strategy:
  matrix:
    ...
  steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Setup environment
      uses: ...
    - name: Run command
      run: |
        echo "Выполняем команду..."
```

Workflows of CPython

Pinned

JIT

Lint

Mypy

Tail calling interpreter

Tests

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled

Workflows of CPython

Pinned - это обязательные проверки, которые должны пройти перед merge Pull Request'a.

JIT

Lint

Mypy

Tail calling interpreter

Tests

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled

jit.yml

узконаправленный CI, который запускается только когда меняются файлы, связанные с JIT компилятором.

```
name: JIT
on:
  pull_request:
    paths: &paths
    - '**jit**'
    - 'Python/bytecodes.c'
    - 'Python/optimizer*.c'
    ...
    - '!**/*.md' #игнорируем документацию
  ...

#отменяет старые запуски
concurrency:
  group: ${{ github.workflow }}-${{
github.head_ref || github.run_id }}
  cancel-in-progress: true
  ...

jobs:
  #tier-2 интерпритатор без JIT компиляции
  interpreter:
    name: Interpreter (Debug)
    runs-on: ubuntu-24.04
    ...
```

```
#matrix на 3 архитектуры
windows:
  name: ${{ matrix.target }}...
  runs-on: ${{ matrix.runner }}
  ...
  target:
    - i686-pc-windows-msvc/msvc
    - x86_64-pc-windows-msvc/msvc
    - aarch64-pc-windows-msvc/msvc
  ...
macos:
  name: ${{ matrix.target }}...
  runs-on: ${{ matrix.runner }}
  ...
linux:
  name: ${{ matrix.target }}...
  runs-on: ${{ matrix.runner }}
  ...

#с тремя сборками free-threading, JIT, tail-call
linux-extras:
  name: ${{ matrix.name }}
  runs-on: ubuntu-24.04
  ...
```

Workflows of CPython

Pinned - это обязательные проверки, которые должны пройти перед merge Pull Request'a.

lint.yml

проверки стиля написанного кода

JIT

Lint

Mypy

Tail calling interpreter

Tests

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled

```
name: Lint

on: [push, pull_request, workflow_dispatch]

permissions:
  contents: read

env:
  FORCE_COLOR: 1
  RUFF_OUTPUT_FORMAT: github

concurrency:
  group: ${{ github.workflow }}-${{ github.head_ref || github.run_id }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    timeout-minutes: 10

    steps:
      - uses: actions/checkout@de0fac2e4500dabe0009e67214ff5f5447ce83dd # v6.0.2
        with:
          persist-credentials: false
      - uses: j178/prek-action@0bb87d7f00b0c99306c8bcb8b8beba1eb581c037 # v1.1.1
```

Workflows of CPython

Pinned - это обязательные проверки, которые должны пройти перед merge Pull Request'a.

JIT

Lint

Mypy

Tail calling interpreter

Tests

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled

mypy.yml

проверка типов критически
важных файлов

```
name: mypy

on:
  push:
    branches:
      - main
  pull_request:
    paths:
      - ".github/workflows/mypy.yml"
      - "Lib/_colorize.py"
      - "Lib/_pyrepl/**"
      - "Lib/test/libregrtest/**"
      ...
```

```
jobs:
  mypy:
    name: Run mypy on ${matrix.target}
    runs-on: ubuntu-latest
    timeout-minutes: 10
    strategy:
      fail-fast: false
    matrix:
      target: [
        "Lib/_pyrepl",
        "Lib/test/libregrtest",
        "Lib/tomllib",
        "Tools/build",
        ...
```

Workflows of CPython

Pinned - это обязательные проверки, которые должны пройти перед merge Pull Request'a.

JIT

Lint

Mypy

Tail calling interpreter

Tests

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled

tail-call.yml

проверяет новую оптимизацию ускорения вызовов функций

```
name: Tail calling interpreter
on:
  pull_request:
    paths: &paths
    - '.github/workflows/tail-call.yml'
    - 'Python/bytecodes.c'
    - 'Python/ceval.c'
    ...

jobs:
  macos:
    name: ${{ matrix.target }}
    runs-on: ${{ matrix.runner }}
    ...

  linux:
    name: ${{ matrix.target }}
    runs-on: ${{ matrix.runner }}
    ...
```

Workflows of CPython

Pinned - это обязательные проверки, которые должны пройти перед merge Pull Request'a.

JIT

Lint

Mypy

Tail calling interpreter

Tests

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled

build.yml

основной CI который запускает все билды и проверки

```
name: Tests
on:
  workflow_dispatch:
  push:
    branches:
      - 'main'
      - '3.*'
  pull_request:
    branches:
      - 'main'
      - '3.*'
jobs:
  #Тесты
  build-context:
    ...

  check-generated-files:
    ...

  check-c-api-docs:
    ...
```

```
#Сборки
build-windows:
...
build-macos:
...
build-ubuntu:
...
build-ubuntu-ssltests:
...

#Финальная проверка на то что все работы прошли
all-required-green:
name: All required checks pass
runs-on: ubuntu-latest
timeout-minutes: 5
needs:
- build-context
- check-docs
- check-autoconf-regen
...
```

Workflows of CPython

Pinned

Build blocks for CI

Это модульные workflow, которые можно вызывать из других workflow.
Это позволяет не дублировать код и легко поддерживать конфигурации.

- Reusable build context
- Reusable C API Docs Check
- Reusable CIFuzz
- Reusable Sanitizer

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled

Workflows of CPython

Pinned

Build blocks for CI

Security & Quality - Группа workflow, отвечающая за безопасность и качество кода.

- CodeQL - статический анализ кода на уязвимости
- Coverage - измерение покрытия тестами
- Verify bundled libexpat - проверка безопасности bundled библиотеки expat
- Verify bundled pip and setuptools - проверка версий и безопасности pip/setuptools

Documentation & Maintenance

Automation & Notification

Disabled

Workflows of CPython

Pinned

Build blocks for CI

Security & Quality

Documentation & Maintenance - отвечает за документацию и поддержание актуальности служебных файлов.

- Docs - сборка и проверка документации
- Generate Lib/idlelib/help.html - генерация help-файла для IDLE
- Read the Docs PR preview - создание превью документации для PR

Automation & Notification

Disabled

Workflows of CPython

Pinned

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification - Группа вспомогательных workflow, которые автоматизируют рутинные задачи и уведомления.

- `Add issue header` - автоматически добавляет заголовок в новые issues
- `Check labels` - проверяет правильность лейблов на PR
- `Mark stale pull requests` - помечает неактивные PR как устаревшие
- `new-bugs-announce notifier` - уведомляет о новых багах
- `Codespaces Prebuilds` - предсборка Codespaces

Disabled

Workflows of CPython

Pinned

Build blocks for CI

Security & Quality

Documentation & Maintenance

Automation & Notification

Disabled - Эти workflow-файлы физически находятся в репозитории, но не используются в основном процессе CI

- Reusable Docs
- Reusable macOS
- Reusable Ubuntu
- Reusable WASI
- Reusable Windows
- Reusable Windows MSI

Pull Request

01

Создание PR

Разработчик открывает PR из форка. CLA-бот проверяет подписание лицензионного соглашения.

Pull Request

01

Создание PR

Разработчик открывает PR из форка. CLA-бот проверяет подписание лицензионного соглашения.

02

Автоматический запуск CI

GitHub Actions запускает lint, build и тесты на Linux / macOS / Windows параллельно.

Pull Request

01

Создание PR

Разработчик открывает PR из форка. CLA-бот проверяет подписание лицензионного соглашения.

02

Автоматический запуск CI

GitHub Actions запускает lint, build и тесты на Linux / macOS / Windows параллельно.

03

Code Review

Core developer проверяет код. Правила требуют одобрения минимум одного мейнтейнера.

Pull Request

01

Создание PR

Разработчик открывает PR из форка. CLA-бот проверяет подписание лицензионного соглашения.

02

Автоматический запуск CI

GitHub Actions запускает lint, build и тесты на Linux / macOS / Windows параллельно.

03

Code Review

Core developer проверяет код. Правила требуют одобрения минимум одного мейнтейнера.

04

Merge

Merge только при зелёных чеках

Pull Request

01

Создание PR

Разработчик открывает PR из форка. CLA-бот проверяет подписание лицензионного соглашения.

02

Автоматический запуск CI

GitHub Actions запускает lint, build и тесты на Linux / macOS / Windows параллельно.

03

Code Review

Core developer проверяет код. Правила требуют одобрения минимум одного мейнтейнера.

04

Merge

Merge только при зелёных чеках

05

BuildBot

После merge BuildBot прогоняет расширенные тесты. При падении — issue и уведомление автора.

Моя практическая часть

Учебный проект, воспроизводящий ключевые практики CPython в миниатюре

Link

Выводы

CI/CD — это не просто «тесты», а центральная часть культуры качества проекта.

Для крупных проектов эффективна двухуровневая архитектура: быстрый CI + глубокое тестирование.

Специализация проверок (JIT, Tail-call, Lint и т.д.) значительно лучше, чем один большой workflow.

Модульность и переиспользование кода (Reusable Workflows) — ключ к поддерживаемости.

Строгие правила («Keep the CI green») реально работают и защищают качество проекта.

Лучшие практики

Обязательные проверки

Lint, mypy, Tests,

Фильтрация путей

Запуск только нужных workflow

Переиспользуемые Workflow

Модульные Workflow

Управление параллелизмом

`cancel-in-progress: true`

Строгая защита веток

Merge только при всех зелёных checks

Matrix подход

Тестирование множества ОС и конфигураций

Отдельные CI для чувствительных фичей

JIT и Tail calling имеют свои workflow

Автоматизация рутины

Labels, stale PRs, issue headers

Спасибо за внимание